

# Projeto: Cálculo de Todos os Pares de Caminhos Mínimos em GPU com CUDA - Entrega até 24/11

## Trabalho 3 - Programação Paralela

November 5, 2025

### 1 Descrição

Problemas de caminhos mínimos em grafos ponderados aparecem em inúmeras aplicações do mundo real — como análise de redes sociais, roteamento de tráfego, redes de comunicação, logística e biologia computacional. Quando o grafo possui milhares de vértices e arestas, a execução de algoritmos clássicos (como Floyd–Warshall, Usando uma versão de Multiplicação de Matrizes ou Dijkstra repetido para cada vértice) em CPU se torna extremamente custosa. A utilização de GPU (Graphics Processing Unit), através da arquitetura CUDA, permite explorar o paralelismo massivo para acelerar esses cálculos. **Este projeto será realizado por grupos de até duas pessoas**, onde cada grupo deverá implementar sua solução paralela utilizando CUDA e a linguagem C.

### 2 Objetivos

- Implementar uma aplicação em CUDA e C capaz de calcular todos os pares de caminhos mínimos (APSP) em um grafo ponderado.
- Avaliar o desempenho da versão GPU em comparação com uma versão sequencial em CPU.
- Realizar experimentos em uma máquina equipada com GPU NVIDIA com pelo menos 16 GB de memória.

### 3 Descrição da Aplicação

- O programa deve gerar ou ler um grafo ponderado com entre 1024 e 4096 vértices.
- O grafo pode ser representado por uma matriz de adjacência (dimensão  $N \times N$ ), onde valores 0 na diagonal indicam ausência de custo e valores  $\infty$  indicam ausência de aresta.
- O programa deve calcular as distâncias mínimas entre todos os pares de vértices, produzindo uma matriz de distâncias mínimas como saída.
- O algoritmo base pode ser o Floyd–Warshall paralelizado com CUDA, a versão de paralela de Multiplicação de Matrizes ou o Dijkstra paralelo.

### Exemplos

#### Exemplo de entrada (grafo.txt)

Grafo com 4 vértices representado por matriz de adjacência:

```
0 3 INF 7
8 0 2 INF
5 INF 0 1
2 INF INF 0
```

(onde INF é substituído por um número grande, como INT\_MAX)

### Saída esperada (distancias.txt)

Matriz com todas as distâncias mínimas calculadas:

```
0 3 5 6
5 0 2 3
3 6 0 1
2 5 7 0
```

## 4 Experimentos

O Grupo deve gerar e processar grafos com tamanhos 1024, 2048, 3072 e 4096 vértices, com densidades de arestas variadas (ex.: 25%, 50%, 75%). Em seguida, medir o tempo total de execução das versões Sequencial (CPU) e paralela em GPU (CUDA).

Após a execução, apresente gráficos e tabelas do tipo tempo de execução  $\times$  tamanho do grafo e speedup  $\times$  tamanho do grafo, além disso, faça uma análise do tempo total de execução, speedup obtido em relação à versão sequencial, eficiência e escalabilidade.

## 5 O Que Entregar

- Código-fonte da solução CUDA e das versões comparativas (sequencial e paralela). Importante descrever os autores em cada código-fonte.
- Scripts de geração dos grafos de teste.
- Relatório técnico contendo:
  1. Descrição da representação do grafo e do algoritmo escolhido.
  2. Estratégias de paralelização (mapeamento de threads, blocos e acesso à memória global e compartilhada).
  3. Resultados experimentais (tabelas e gráficos).
  4. Comparação detalhada entre as abordagens CPU sequencial, e paralela em GPU.
  5. Discussão sobre o desempenho, limitações e possíveis otimizações.
- Cada membro do grupo deve gerar e entregar um único arquivo .tgz com o comando dado a seguir e depois submeter este arquivo no AVA:

```
tar czvf felca.pereira.tgz APSP.cu relatorio.pdf
```

onde `tar` é um utilitário de arquivamento do Linux, `felca.pereira` é o seu login e `.tgz` é uma extensão que identifica este arquivo.

Bom Trabalho!